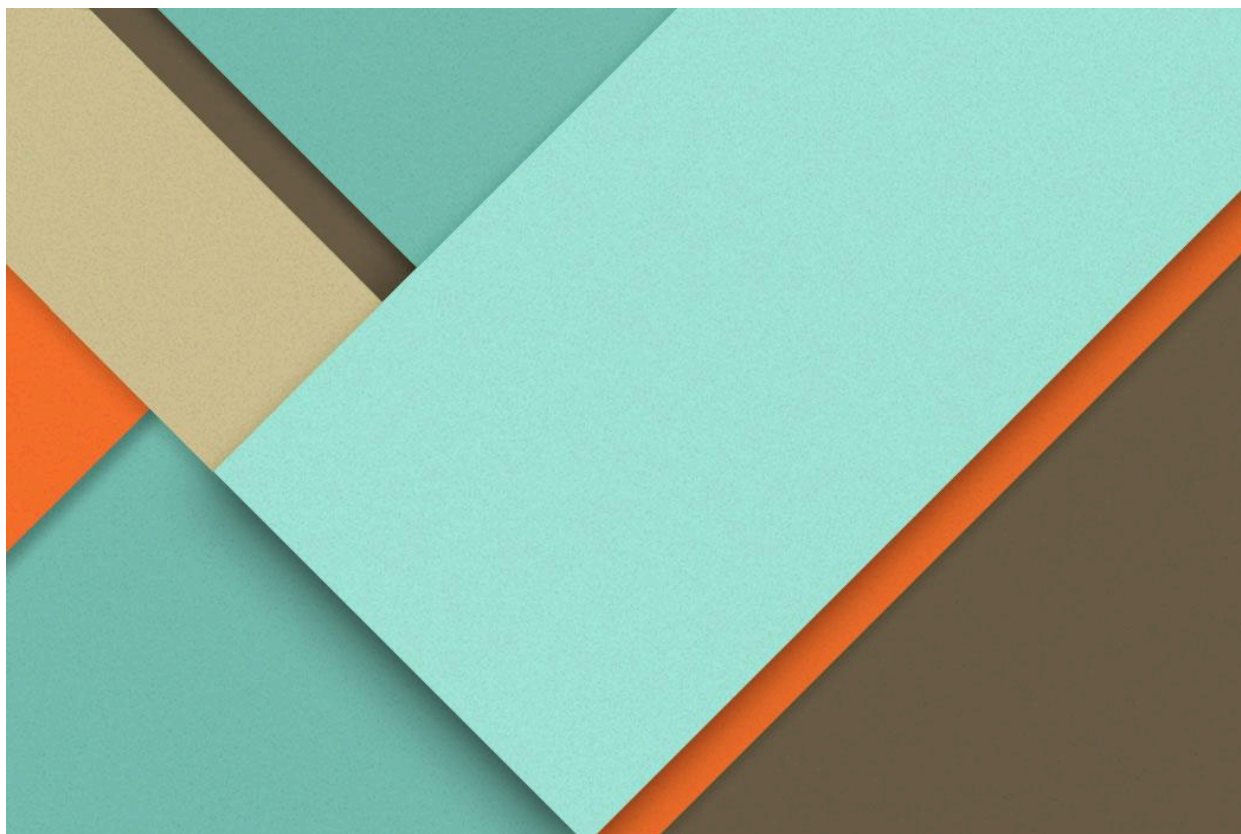




Rapport projet Data

OUTALEB Manel

Elève de B2 - Ynov Campus Casablanca

Introduction	2
Organisation du code et Architecture Strategy	2
I. Logique Métier (Le Modèle de Nettoyage et de Calcul)	3
Présentation des phases de développement	3
II. Phase 1 – Structure et Audit Initial	3
III. Phase 2 – Logique Métier et Algorithmes de Nettoyage	3
IV. Phase 3 & 4 – Visualisation et Exploration (Vues)	3
V. Phase 5 – Storytelling et Traçabilité	4
Explication de code + captures	4
1. Interaction et Nettoyage des Données	4
2. Exemple d'affichage des calculs	4
3. Gestion des anomalies et Sécurité des Données	5
Conclusion du rapport	5

Introduction

Ce projet consiste en la conception et le développement d'un pipeline d'analyse de données exploratoire pour le réseau de distribution MarketCorp: l'objectif est de créer un processus robuste capable d'auditer, de nettoyer et d'analyser des données de ventes hétérogènes pour en tirer des recommandations stratégiques.

Le programme applique une architecture de traitement de données séquentielle (ETL - Extraction, Transformation, Visualisation) pour garantir la lisibilité et la maintenabilité du code sous un environnement interactif Google Colab.

Organisation du code et Architecture Strategy

Le projet est organisé selon une structure modulaire divisée en plusieurs blocs logiques au sein du notebook :

- **Extraction & Audit:** Importation des données brutes et diagnostic de l'état de la base (valeurs manquantes, types de données).
- **Transformation:** Utilisation des bibliothèques Pandas et NumPy pour le nettoyage, l'uniformisation et la création de sous-ensembles de données.
- **Visualisation:** Utilisation de Matplotlib pour générer des tableaux de bord dynamiques (histogrammes, nuages de points, ...) de manière claire et aérée.

I. Logique Métier (Le Modèle de Nettoyage et de Calcul)

Le cœur du système repose sur des règles de gestion des données strictes pour éviter tout biais statistique :

- **Traitement des Poids (Item_Weight):** Remplacement dynamique des valeurs manquantes par la moyenne globale via `np.nanmean`, garantissant la préservation de la distribution.
- **Uniformisation des Catégories:** Standardisation des labels textuels (ex: fusion de "LF", "low fat" vers "Low Fat") via des opérateurs logiques pour un regroupement précis.
- **Calculs Comparatifs:** Segmentation et agrégation des données pour comparer les performances selon la taille du magasin ou le prix de vente maximal (MRP).

Présentation des phases de développement

Le développement a été structuré en phases logiques pour assurer la conformité aux exigences de l'analyse métier et une organisation rigoureuse du code Python.

II. Phase 1 – Structure et Audit Initial

Cette phase a consisté à poser les fondations techniques du projet pour garantir la compréhension des données entrantes.

- **Environnement:** Configuration du notebook Google Colab et importation des librairies essentielles (`pandas`, `numpy`, `matplotlib.pyplot`).
- **Diagnostic:** Mise en place d'un audit rapide (`shape`, `dtypes`, `describe()`) ⇒ cette étape permet au système de détecter instantanément les anomalies (données manquantes).

III. Phase 2 – Logique Métier et Algorithmes de Nettoyage

Cette étape constitue le cerveau du pipeline, là où toutes les règles de correction ont été centralisées.

- **Imputation Fiable:** Développement d'algorithmes traitant les valeurs nulles sans recourir aux fonctions basiques (`fillna`), privilégiant la méthode `.replace()` et `np.where()`.
- **Exactitude des Catégories:** Remplacement des visibilitées impossibles par la médiane globale ⇒ chaque correction a été validée par `isnull().sum()` pour assurer une base saine.

IV. Phase 3 & 4 – Visualisation et Exploration (Vues)

L'objectif de cette phase était de transformer les données brutes en une interface visuelle intuitive pour l'aide à la décision.

- **Design en "Dashboard":** Utilisation de `plt.subplot` et `tight_layout` pour créer une mise en page aérée sous forme de mini-tableau de bord ⇒ cette structure hiérarchise l'information (distribution, répartition, corrélation).

- **Analyse Multidimensionnelle:** Mise au point de graphiques croisant le prix et les ventes, offrant ainsi une rétroaction visuelle immédiate sur les leviers de performance.

V. Phase 5 – Storytelling et Traçabilité

La dernière phase assure la transition d'un simple script de calcul vers un outil stratégique complet.

- **Synthèse Métier :** Traduction des calculs Python en recommandations concrètes (identification des produits performants et des magasins en difficulté).
- **Planification :** Élaboration d'un plan d'action sur 30 jours basé sur les faits chiffrés, garantissant un suivi administratif précis.

Explication de code + captures

Cette section détaille l'implémentation technique des fonctionnalités clés.

1. Interaction et Nettoyage des Données

Le système manipule les données via des opérations vectorisées pour une performance optimale:

- `np.where` : Applique des conditions complexes en temps réel.
- `value_counts` : Compte la répartition des catégories.

```
# Item_Fat_Content : Uniformisation avec l'opérateur logique | (OU)
df['Item_Fat_Content'] = np.where(
    (df['Item_Fat_Content'] == 'low fat') | (df['Item_Fat_Content'] == 'LF'),
    'Low Fat',
    df['Item_Fat_Content']
)
df['Item_Fat_Content'] = df['Item_Fat_Content'].replace('reg', 'Regular')

print("--- RÉPARTITION PAR TYPE DE MAGASIN ---")
print(df['Outlet_Type'].value_counts())
print("\n")
```

2. Exemple d'affichage des calculs

Lors de l'analyse des performances par taille de magasin, le système décompose :

- Filtre sur les petits magasins.
- Extraction de la colonne cible.
- Calcul de la moyenne.
- Comparaison formatée des résultats.

```
# 3. Comparaison filtrée
print("--- ANALYSE COMPARATIVE DES VENTES ---")
moy_petit = df[df['Outlet_Size'] == 'Small']['Item_Outlet_Sales'].mean()
moy_grand = df[df['Outlet_Size'] == 'High']['Item_Outlet_Sales'].mean()
```

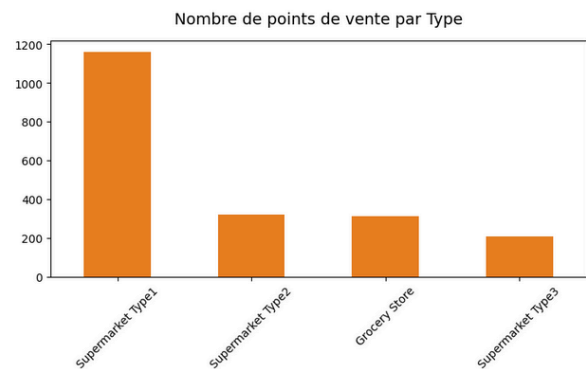
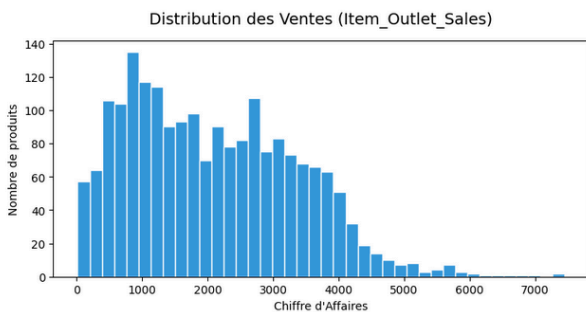
3. Gestion des anomalies et Sécurité des Données

Le système gère de manière sécurisée les incohérences de la base de données :

- **Valeurs Aberrantes:** Utilisation du Boxplot dans Matplotlib pour identifier visuellement les extrêmes de ventes sans altérer le dataset original.
- **Données Manquantes:** Traitement systématique des NaN (par la moyenne, la médiane ou "Inconnu") pour éviter le crash des algorithmes ou le biais des graphiques.

```
# Item_Weight : Remplacement par la moyenne
moyenne_poids = np.nanmean(df['Item_Weight'])
df['Item_Weight'] = df['Item_Weight'].replace(np.nan, moyenne_poids)

# Outlet_Size : Remplacement par "Inconnu"
df['Outlet_Size'] = df['Outlet_Size'].replace(np.nan, "Inconnu")
```



Conclusion du rapport

La réalisation de ce projet, sous Google Colab, a permis :

- D'appliquer strictement une logique de pipeline Data en Python.
- De maîtriser l'utilisation de Pandas et NumPy pour le traitement massif de données.
- De développer un moteur d'analyse flexible et visuellement percutant.
- De séparer le traitement brut des données (nettoyage) de l'interprétation métier.